# SPARQL 1.2 Query Results CSV and TSV Formats

W3C Working Draft 14 August 2025




**▼ More details about this document**

**This version:**

https://www.w3.org/TR/2025/WD-sparql12-results-csv-tsv-20250814/

**Latest published version:**

https://www.w3.org/TR/sparql12-results-csv-tsv/

**Latest editor's draft:**

https://w3c.github.io/sparql-results-csv-tsv/spec/

**History:**

https://www.w3.org/standards/history/sparql12-results-csv-tsv/

Commit history

**Test suite:**

https://w3c.github.io/rdf-tests/

**Latest Recommendation:**

https://www.w3.org/TR/2013/REC-sparql11-results-csv-tsv-20130321

**Editors:**

Ruben Taelman

Gregory Williams

Thomas Pellissier Tanon

**Former editor:**

Andy Seaborne

**Feedback:**

GitHub w3c/sparql-results-csv-tsv (pull requests, new issue, open issues)

public-rdf-star-wg@w3.org with subject line `[sparql12-results-csv-tsv] … message topic …` (archives)

Copyright © 2012-2025 World Wide Web Consortium. W3C® liability, trademark and permissive document license rules apply.

## Abstract

The formats CSV [*RFC4180*] (comma separated values) and TSV [*IANA-TSV*] (tab separated values) provide simple, easy to process formats for the transmission of tabular data. They are supported as input datat formats by many tools, particularly spreadsheets. This document describes their use for expressing SPARQL query results from `SELECT` queries.


## Status of This Document

*This section describes the status of this document at the time of its publication. A list of current W3C publications and the latest revision of this technical report can be found in the W3C standards and drafts index.*

This specification is published by the RDF Star Working Group as part of the update of specifications for format and errata.

This document was published by the RDF & SPARQL Working Group as a Working Draft using the Recommendation track.

Publication as a Working Draft does not imply endorsement by W3C and its Members.

This is a draft document and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to cite this document as other than a work in progress. Future updates to this specification may incorporate new features.

This document was produced by a group operating under the W3C Patent Policy. W3C maintains a public list of any patent disclosures made in connection with the deliverables of the group; that page also includes instructions for disclosing a patent. An individual who has actual knowledge of a patent that the individual believes contains Essential Claim(s) must disclose the information in accordance with section 6 of the W3C Patent Policy.

This document is governed by the 03 November 2023 W3C Process Document.

## § Set of Documents

This document is one of twelve SPARQL 1.2 documents produced by the RDF & SPARQL Working Group.

▶ **List of documents**

# Table of Contents


**Abstract**

**Status of This Document**
Set of Documents

**1. Introduction**
1.1 Example

**2. Transmission issues using CSV and TSV Formats**

**3. CSV — Comma Separated values**
3.1 Serializing the Results Table
3.2 Serializing RDF Terms
3.3 Example of CSV-Serialized Results
3.4 Example of CSV-Serialized Results with Triple Terms

**4. TSV — Tab Separated values**
4.1 Serializing the Results Table
4.2 Serializing RDF Terms
4.3 Example of TSV-Serialized Results
4.4 Example of TSV-Serialized Results with Triple Terms

**5. Conformance**
5.1 Conformance

**A. Changes between SPARQL 1.1 Query Results CSV and TSV Formats and SPARQL 1.2 Query Results CSV and TSV Formats**

**B. Privacy Considerations**

**C. Security Considerations**

**D.** **Internationalization Considerations**

**E.** **Index**
E.1 Terms defined by this specification
E.2 Terms defined by reference

**F.** **References**
F.1 Normative references
F.2 Informative references

# 1. Introduction

This document describes CSV and TSV formats for expressing the results of a SPARQL `SELECT` query. They provide lowest common denominator formats between systems using different implementation technologies.

Other formats for expression of SPARQL results are the SPARQL Results XML Format [*SPARQL12-RESULTS-XML*] and the SPARQL Results JSON Format [*SPARQL12-RESULTS-JSON*]. Each format is useful in different application scenarios.

The SPARQL Results CSV Format is a lossy encoding of a table of results. It does not encode all the details of each RDF term in the results; instead, it just gives a string without indicating the type of the term (IRI, Literal, Literal with datatype, Literal with language, or blank node). This makes it simple to consume data, such as text and numbers, in applications that don't need to understand the details of RDF. In some applications, guesses as to which elements are hyperlinks are made pragmatically, for example, guessing that strings starting "`http://`" are links.

The SPARQL Results TSV Format does encode the details of RDF terms in the results table, by using the syntax that SPARQL [*SPARQL12-QUERY*] and Turtle [*RDF12-TURTLE*] use. An application receiving a TSV-encoded result set can split each line into elements of the result row, and extract all the details of the RDF terms it wishes to process by simple string processing, without a complete XML or JSON parser as may by required by the more complex SPARQL result formats.

When this document uses the words *MUST*, *MUST NOT*, *SHOULD*, *SHOULD NOT*, and *MAY*, and the words appear as emphasized text, they must be interpreted as described in [*RFC2119*].

## § 1.1 Example

The following artificial example is used to illustrate the features of serializing results in each format.

| **x** | **literal** | ***Comment (not part of the table)*** |
|---|---|---|
| <http://example/x> | String | An IRI and a string consisting of characters S-t-r-i-n-g |
| <http://example/x> | String-with-dquote" | String with a double quote in it. |
| _:b0 | Blank node | Blank node |
| | Missing 'x' | No RDF term for the **x** column |
| | | This row has no terms in it. |
| <http://example/x> | | No term in the **literal** column. |
| _:b1 | "String-with-lang"@en | An RDF literal with a language tag |
| _:b1 | "String-with-lang-dir"@en--ltr | An RDF literal with a directional language tag |
| _:b1 | 123 | An RDF literal, datatype xsd:integer, and lexical form 123. |

The following artificial example is used to illustrate the features of serializing results containing triple terms.

| **x** | **triple** | ***Comment (not part of the table)*** |
|---|---|---|
| "Alice" | <<( <http://example/alice> <http://example/knows> <http://example/bob> )>> | A plain string and triple term with subject IRI `http://example/alice`, predicate IRI `http://example/knows`, and object IRI `http://example/bob`. |
| "Bob" | <<( <http://example/bob> <http://example/knows> <http://example/alice> )>> | A plain string and triple term with subject IRI `http://example/bob`, predicate IRI `http://example/knows`, and object IRI `http://example/alice`. |

| | | |
|---|---|---|
| "Carol" | <<( <http://example/carol> <http://example/says> "Hello world, my name is \"Alice\"." )>> | A plain string and triple term with subject IRI `http://example/carol`, predicate IRI `http://example/says`, and object literal `Hello world, my name is "Alice".`. |

## § 2. Transmission issues using CSV and TSV Formats

The SPARQL result formats described here conform to the formal specifications of the relevant formats, Comma Separated values (CSV) [*RFC4180*] and Tab Separated Value (TSV) [*IANA-TSV*].

Systems providing these formats should note that the content types are `text/csv` for CSV and `text/tab-separated-values` for TSV. Being `text/*`, the default character set is US-ASCII. The `charset` parameter *SHOULD* be used in conjunction with SPARQL Results; UTF-8 is recommended; giving us `text/csv; charset=utf-8` and `text/tab-separated-values; charset=utf-8`.

The end-of-line in CSV is `CRLF`, i.e., Unicode codepoints 13 (`U+000D`) and 10 (`U+000A`).

The end-of-line in TSV is `EOL`, i.e., Unicode codepoint 10 (`U+000A`).

Applications reading these formats are advised to cope with both CRLF and LF as end of line markers and not rely on conformance to the formal specifications.

## § 3. CSV — Comma Separated values

In the SPARQL Results CSV Format, the results table is serialized as one line listing the variables in the results, using the CSV header line, followed by one line for each query solution. (Note: a line may end up split by newlines in the data). Values in the results are:

- strings for
  - URIs
  - lexical forms of non-numeric XSD datatypes

    - blank node labels
    - triple terms
- numbers for literals of numeric XSD datatypes

## § 3.1 Serializing the Results Table

The first line of a SPARQL Results CSV Format response is the header line, giving the names of the variables used in the result set. The header line consists of the variable names, without leading question marks `?`, separated by commas.

While the `text/csv` format does not require a header row, the SPARQL Results CSV Format *MUST* use a header row. If the content type parameter `header` is used, it *MUST* be `header=present`.

The remaining rows are the values of the results, with each binding determined by the position in the row, corresponding to the entry in the header line.

If a variable is not bound, an empty field is used (e.g. `,,`). Each row *MUST* have the same number of fields, with each field corresponding to a binding to the variable in the header line in the same field position.

## § 3.2 Serializing RDF Terms

The entry in each field is the string corresponding to the RDF term value. (cf. SPARQL `STR()`) without syntax to denote what kind of term it is. The encoding quoting rules of CSV format must be used.

Blank nodes use the `_:label` form from Turtle and SPARQL. Use of the same label indicates the same blank node within the result set but has no significance outside the result set.

Triple terms are enclosed in `<<( … )>>`, as `<<( subject predicate object )>>`, where the *subject*, *predicate*, and *object* terms are recursively serialized. The serializations of *subject*, *predicate*, and *object MUST* be separated by a single space character (SPACE, code point 32,

`U+0020`). The single space character preceding *subject*, and the single space character following *object* are optional, and are mainly valuable for human readability. If the *object* is a literal, then it *MUST* be encapsulated within a pair of quotation marks `""`.

Fields containing any of `"` (QUOTATION MARK, code point 34, `U+0022` in Unicode [*UNICODE*]), `,` (COMMA, code point 44, `U+002C`), `LF` (code point 10, `U+000A`), or `CR` (code point 13, `U+000D`) *MUST* be quoted using the quoting mechanism of RFC4180 [*RFC4180*]. Fields are delimited by a pair of quotation marks `"` (code point `U+0022`). Within quoted strings, all characters except `"`, including new line characters, have their exact meaning — newlines do not end a CSV record. Inline `"` is written using a pair of quotation marks `""`. This quoting mechanism is applied recursively for terms in triple terms.

> NOTE
>
> Since literals in the object position of triple terms are encapsulated within a pair of quotation marks `"`, and the full triple term is always encapsulated within quotation marks as well, this requires the subsequent escaping of the "inner" quotation marks encapsulating the literal. Such cases will result in triple terms in the form of `"<<( subject predicate ""object-literal"" )>>"`.

The standard CSV format does not distinguish between missing values and empty strings. The SPARQL 1.2 Results CSV Format uses the same representation for unbound variables as for variables bound to an empty string literal. The other SPARQL Result formats (based on JSON, TSV, or XML) can be used if this distinction is required.

## § 3.3 Example of CSV-Serialized Results

```
x,literal
http://example/x,String
http://example/x,"String-with-dquote"""
_:b0,Blank node
,Missing 'x'
,
http://example/x,
_:b1,String-with-lang
_:b1,String-with-lang-dir
_:b1,123
```

### § 3.4 Example of CSV-Serialized Results with Triple Terms

```
x,triple
"Alice",<<( http://example/alice http://example/knows http://exampl
"Bob",<<( http://example/bob http://example/knows http://example/al
"Carol","<<( http://example/carol http://example/says ""Hello world
```

## § 4. TSV — Tab Separated values

In the SPARQL Results TSV Format, the results table is serialized as one line listing the variables in the results, followed by one line for each query solution. All RDF terms used in the format are encoded in the format specified by Turtle [RDF12-TURTLE] except that the triple term forms for the lexical part of literals *MUST NOT* be used. These forms would allow raw newlines and tabs that are part of the TSV format. A TSV format SPARQL result set must use the single quoted literal forms, together with any necessary escapes such as `\t`, `\n`, and `\r`.

### § 4.1 Serializing the Results Table

The results table is serialized as one line listing the variables in the results, followed by one line for each query solution. This first line is required by the TSV format [*IANA-TSV*], unlike CSV, where it is optional.

Variables are serialized in SPARQL syntax, using question mark `?` character followed by the variable name.

Each row of the result set is serialized by sequence of RDF terms in SPARQL syntax, separated by a tab (Horizontal Tab, Unicode code point `U+0009`) character.

If a variable is not bound in a row, an empty field is used. Each row *MUST* have the same number of fields, corresponding to the variables listed in the first row.

## § 4.2 Serializing RDF Terms

The SPARQL Results TSV Format serializes RDF terms in the results table by using the syntax that SPARQL [*SPARQL12-QUERY*] and Turtle [*RDF12-TURTLE*] use.

IRIs are enclosed in `<...>`, literals are enclosed with double quotes `"..."` or single quotes `' ...'` with optional `@lang` or `^^` for datatype. IRIs are written enclosed in `<...>`. They *MUST* conform to the IRI rule of *Internationalized Resource Identifiers (IRIs)*. Such IRIs include the IRI scheme and *MUST NOT* be Relative Reference. This includes IRIs used as datatypes.

Literals are written with the lexical form in quotes. Tab, newline, and carriage return characters (Unicode code points `U+0009` (tab), `U+0010` (line feed) and `U+0013` (carriage return)) are encoded in strings as `\t`, `\n` and `\r` respectively. The long string forms using triple quotes — `"""` or `'''` — *MUST NOT* be used.

The abbreviated forms for numbers (XSD integers, decimals, and doubles) *SHOULD* be used.

Blank nodes use the `_:label` form from Turtle and SPARQL. Use of the same label indicates the same blank node within the result set, but has no significance outside the result set.

Triple terms are enclosed in `<<( … )>>`, as in `<<(subject predicate object)>>`, where the *subject*, *predicate*, and *object* terms are recursively serialized. The serializations of *subject*, *predicate*, and *object* *MUST* be separated by a single space character (SPACE, code point 32,

U+0020). The single space character preceding *subject*, and the single space character following *object* are optional, and are mainly valuable for human readability.

## § 4.3 Example of TSV-Serialized Results

Writing <TAB> for a raw tab character (Unicode code point U+0009):

```
?x<TAB>?literal
<http://example/x><TAB>"String"
<http://example/x><TAB>"String-with-dquote\""


_:blank0<TAB>"Blank node"
<TAB>"Missing 'x'"
<TAB>
<http://example/x><TAB>
_:blank1<TAB>"String-with-lang"@en
_:blank1<TAB>"String-with-lang-dir"@en--ltr
_:blank1<TAB>123
```

## § 4.4 Example of TSV-Serialized Results with Triple Terms

Writing <TAB> for a raw tab character (Unicode code point U+0009):

```
?x<TAB>?triple
"Alice"<TAB><<( <http://example/alice> <http://example/knows> <http
"Bob"<TAB><<( <http://example/bob> <http://example/knows> <http://e
"Carol"<TAB><<( <http://example/carol> <http://example/says> "Hello
```

# § 5. Conformance

As well as sections marked as non-normative, all authoring guidelines, diagrams, examples, and notes in this specification are non-normative. Everything else in this specification is normative.

The key words *MAY*, *MUST*, *MUST NOT*, *SHOULD*, and *SHOULD NOT* in this document are to be interpreted as described in BCP 14 [*RFC2119*] [*RFC8174*] when, and only when, they appear in all capitals, as shown here.

### § 5.1 Conformance

## § A. Changes between SPARQL 1.1 Query Results CSV and TSV Formats and SPARQL 1.2 Query Results CSV and TSV Formats

*This section is non-normative*.

- Allow triple terms to be expressed in 3.2 Serializing RDF Terms and 4.2 Serializing RDF Terms
- Support directional language-tagged strings in 3.2 Serializing RDF Terms and 4.2 Serializing RDF Terms

## § B. Privacy Considerations

TODO

## § C. Security Considerations

TODO

## § D. Internationalization Considerations

TODO

## § E. Index

### § E.1 Terms defined by this specification

### § E.2 Terms defined by reference

[*RFC3986*] defines the following:

Relative Reference

## § F. References

### § F.1 Normative references


**[IANA-TSV]**
*Definition of tab-separated-values (tsv)*. Paul Lindner. IANA. June 1993. IANA Media Type Registration. URL: https://www.iana.org/assignments/media-types/text/tab-separated-values

**[RDF12-TURTLE]**

*RDF 1.2 Turtle*. Gregg Kellogg; Dominik Tomaszuk. W3C. 12 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-turtle/

**[RFC2119]**

*Key words for use in RFCs to Indicate Requirement Levels*. S. Bradner. IETF. March 1997. Best Current Practice. URL: https://www.rfc-editor.org/rfc/rfc2119

**[rfc3986]**

*Uniform Resource Identifier (URI): Generic Syntax*. T. Berners-Lee; R. Fielding; L. Masinter. IETF. January 2005. Internet Standard. URL: https://www.rfc-editor.org/rfc/rfc3986

**[RFC3987]**

*Internationalized Resource Identifiers (IRIs)*. M. Duerst; M. Suignard. IETF. January 2005. Proposed Standard. URL: https://www.rfc-editor.org/rfc/rfc3987

**[RFC4180]**

*Common Format and MIME Type for Comma-Separated Values (CSV) Files*. Y. Shafranovich. IETF. October 2005. Informational. URL: https://www.rfc-editor.org/rfc/rfc4180

**[RFC8174]**

*Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words*. B. Leiba. IETF. May 2017. Best Current Practice. URL: https://www.rfc-editor.org/rfc/rfc8174

**[SPARQL12-QUERY]**

*SPARQL 1.2 Query Language*. Olaf Hartig; Andy Seaborne; Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 11 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-query/

**[SPARQL12-RESULTS-JSON]**

*SPARQL 1.2 Query Results JSON Format*. Andy Seaborne; Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 8 May 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-results-json/

**[SPARQL12-RESULTS-XML]**

*SPARQL 1.2 Query Results XML Format*. Ruben Taelman; Dominik Tomaszuk; Thomas Pellissier Tanon. W3C. 27 December 2024. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-results-xml/

**[UNICODE]**

*The Unicode Standard*. Unicode Consortium. URL: https://www.unicode.org/versions/latest/

## § F.2 Informative references


**[SPARQL12-CONCEPTS]**

*SPARQL 1.2 Concepts*. The W3C RDF & SPARQL Working Group. W3C. W3C Editor's Draft. URL: https://w3c.github.io/sparql-concepts/spec/

**[SPARQL12-ENTAILMENT]**

*SPARQL 1.2 Entailment Regimes*. Peter Patel-Schneider. W3C. 19 December 2024. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-entailment/

**[SPARQL12-FEDERATED-QUERY]**

*SPARQL 1.2 Federated Query*. Ruben Taelman; Gregory Williams. W3C. 5 May 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-federated-query/

**[SPARQL12-GRAPH-STORE-PROTOCOL]**

*SPARQL 1.2 Graph Store Protocol*. Andy Seaborne; Thomas Pellissier Tanon. W3C. 19 December 2024. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-graph-store-protocol/

**[SPARQL12-NEW]**

*What's New in SPARQL 1.2*. The W3C RDF & SPARQL Working Group. W3C. W3C Editor's Draft. URL: https://w3c.github.io/sparql-new/spec/

**[SPARQL12-PROTOCOL]**

*SPARQL 1.2 Protocol*. Andy Seaborne; Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 24 June 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-protocol/

**[SPARQL12-SERVICE-DESCRIPTION]**

*SPARQL 1.2 Service Description*. Ruben Taelman; Gregory Williams. W3C. 19 December 2024. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-service-description/

**[SPARQL12-UPDATE]**

*SPARQL 1.2 Update*. Ruben Taelman; Andy Seaborne; Thomas Pellissier Tanon. W3C. 6 February 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-update/


↥